

Instituto Politécnico Nacional

Escuela Superior de Cómputo



Ingeniería en Inteligencia Artificial
Academia de Sistemas Distribuidos
Cómputo Paralelo
6BM1

Práctica 1: Plataformas y Herramientas

Flores López Mariana -
Gónzalez Gónzalez Alan -
Pérez Guzmán Shaiel Aylin - 2024630673

Plan 2020
Profesor: Rodrigo Ojeda Santillán
08 de Octubre de 2026

1. **Objetivo**

   Preparar un entorno de trabajo para realizar experimentos de cómputo paralelo, identificar las características del hardware y software disponible, y utilizar herramientas de medición de rendimiento. Construir una línea base secuencial mediante diferentes niveles de optimización y evaluar el rendimiento de CPU y GPU.

2. **Entorno de trabajo**

   El cómputo paralelo es una herramienta fundamental en el procesamiento de grandes cantidades de información, ya que permite aprovechar los recursos de hardware para ejecutar operaciones de manera más eficiente. Su importancia se refleja en áreas como la inteligencia artificial, donde los cálculos numéricos y el procesamiento de matrices son operaciones frecuentes. Sin embargo, para determinar qué tan eficiente es una implementación, es necesario realizar mediciones de rendimiento bajo condiciones controladas.

   En esta práctica se exploran las herramientas y plataformas utilizadas para el cómputo paralelo, comenzando con la configuración del entorno de desarrollo y la identificación de las características del hardware disponible.

   Posteriormente, se establece una implementación secuencial de la multiplicación de matrices como referencia para medir y comparar los tiempos de ejecución obtenidos mediante diferentes niveles de optimización del compilador, Python y NumPy.

   Asimismo, se analiza la importancia de realizar mediciones confiables mediante ejecuciones repetidas,el cálculo del promedio y la desviación estándar.

   Finalmente, se explora la ejecución de operaciones en GPU y se documenta el trabajo realizado mediante el uso de Git y GitHub. De esta manera, la práctica permite comprender cómo las herramientas de software, las características de hardware y las metodologías de medición influyen en el rendimiento de los programas

3. Desarrollo

3.1 Entorno de Trabajo

En esta primera parte se verificaron las herramientas necesarias para realizar la práctica, incluyendo GCC, G++, Python, Git y NumPy. También se comprobó el soporte para OpenMP, con el objetivo de asegurar que el entorno de desarrollo contara con los recursos necesarios para compilar y ejecutar los programas. Esta preparación permitió establecer las condiciones iniciales para realizar las pruebas de rendimiento de las siguientes etapas.

3.2 Ficha técnica del Hardware

En esta sección se verificaron las características del equipo utilizado para realizar las pruebas, incluyendo el procesador, los núcleos disponibles, la memoria RAM y el sistema operativo. Esta información permite conocer los recursos de hardware con los que se cuenta y comprender como pueden influir en el rendimiento de los programas ejecutados. Los resultados obtenidos se presentan en las siguientes tablas.

Campo	Flores López Mariana
Modelos de CPU	AMD Ryzen 7 5700U with Radeon Graphics
Núcleos físicos / hilos lógicos	8 núcleos físicos / 16 hilos lógicos
Frecuencia base / turbo	1.80 GHz / hasta 4.30 GHz
Caché L1d / L2 / L3	L1D: 256 KiB / L2: 4 MiB / L3: 4 MiB
Memoria RAM	7.4 GiB disponibles en WSL (~8 GB instalados)
Extensiones vectoriales	SSE4.2, AVX, AVX2 y FMA
Sistema operativo y kernel	Ubuntu 24.04.5 LTS (WSL2) / Kernel 6.18.40.1-microsoft-standard-WSL2
Compilador	gcc 13.3.0
Python / NumPy / BLAS	Python 3.12.13 / NumPy 2.5.3 / OpenBLAS 0.3.34.106.0
GPU (si hay)	AMD Radeon Graphics integrada; no se detectó GPU NVIDIA mediante `nvidia-smi`
Condiciones	Computadora conectada a la corriente, sin otras aplicaciones de alto consumo abiertas, pruebas ejecutadas en WSL2

Tabla 1. Ficha técnica del primer integrante.

Campo	Gónzalez Gónzalez Alan
Modelos de CPU	AMD Ryzen 7 5700U with Radeon Graphics
Núcleos físicos / hilos lógicos	8 núcleos físicos / 16 hilos lógicos
Frecuencia base / turbo	1.80 GHz / hasta 4.30 GHz
Caché L1d / L2 / L3	L1D: 256 KiB / L2: 4 MiB / L3: 4 MiB
Memoria RAM	7.4 GiB disponibles en WSL (~8 GB instalados)
Extensiones vectoriales	SSE4.2, AVX, AVX2 y FMA
Sistema operativo y kernel	Ubuntu 24.04.5 LTS (WSL2) / Kernel 6.18.40.1-microsoft-standard-WSL2
Compilador	gcc 13.3.0
Python / NumPy / BLAS	Python 3.12.13 / NumPy 2.5.3 / OpenBLAS 0.3.34.106.0
GPU (si hay)	AMD Radeon Graphics integrada; no se detectó GPU NVIDIA mediante `nvidia-smi`
Condiciones	Computadora conectada a la corriente, sin otras aplicaciones de alto consumo abiertas, pruebas ejecutadas en WSL2

Tabla 2. Ficha técnica del segundo integrante.

Campo	Pérez Guzmán Shaiel Aylin
Modelos de CPU	AMD Ryzen 5 5500U with Radeon Graphics
Núcleos físicos / hilos lógicos	6 núcleos físicos / 12 hilos lógicos reportados por WSL2
Frecuencia base / turbo	Pendiente de consultar las especificaciones del modelo
Caché L1d / L2 / L3	L1D: 192 KiB / L2: 3 MiB / L3: 4 MiB
Memoria RAM	6.7 GiB totales visibles en WSL2; 6.2 GiB disponibles en el momento de consulta
Extensiones vectoriales	SSE, SSE2, SSE3 (aparece como pni), SSSE3, SSE4.1, SSE4.2, AVX, AVX2 y FMA
Sistema operativo y kernel	Ubuntu 24.04, ejecutado mediante WSL2; kernel 6.18.40.1-microsoft-standard-WLS2
Compilador	GCC y G++ 13.3.0

Python / NumPy / BLAS	Python 3.12.3. NumPy y BLAS pendiente de verificar. GPU pendiente de verificar, el equipo reporta gráficos integrados AMD Radeon.
GPU (si hay)	Gráficos integrados AMD Radeon (™) Graphics, detectados por Windows. No se ha verificado su disponibilidad para cómputo general desde WSL2. Las pruebas de GPU de la práctica se realizan en Google Colab o Kaggle, utilizando la GPU que se asigne.
Condiciones	Laptop con Windows 11 y Ubuntu 24.04 mediante WSL2. Las mediciones se realizaron en un entorno virtualizado.

Tabla 3. Ficha técnica del tercer integrante del equipo.

3.3 Metodología de medición

Se midió el tiempo de pared, es decir, el tiempo transcurrido entre el inicio y el final de cada operación. En C se utilizó clock_gettime con CLOCK_MONOTONIC; en Python se utilizó time.perf_counter(). Para NumPy también se midió el tiempo de CPU mediante time.process_time().

Se realizó una ejecución de calentamiento que no se incluyó en los resultados y posteriormente cinco repeticiones por configuración. Se calculó el promedio y la desviación estándar. El rendimiento se expresó en GFLOPS mediante la aproximación de (2n^3/t), donde (n) representa la dimensión de la matriz y (t) el tiempo promedio en segundos.

En C se compararon las opciones de compilación -O0, -O2 y -O3 -march=native.
En Python se utilizó una implementación con listas y ciclos anidados. NumPy ejecutó la multiplicación mediante sus rutinas numéricas optimizadas, tanto con la configuración habitual como restringiendo los hilos mediante OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1 y MKL_NUM_THREADS=1.

Para la parte de GPU se utilizó CuPy con matrices de tipo float32, una ejecución de calentamiento y cinco repeticiones. Se compararon los tiempos de CPU y GPU para matrices de 256 x 256 y 4096 x 4096. En la medición de GPU sincronizada se esperó a que terminara la operación antes de detener el cronómetro.

3.4 Resultados

3.4.1 Tabla resumen

Los tiempos se expresan en segundos y GFLOPS representa el rendimiento promedio reportado por los programas.

Variante	n	Promedio (s)	Desv. estándar (s)	GFLOPS
C -O0	256	0.146838	0.012236	0.22980
C -O0	512	1.315971	0.016808	0.20420
C -O0	1024	26.504649	0.736820	0.08120
C -O2	256	0.078541	0.002225	0.42740
C -O2	512	1.843508	0.010402	0.14560
C -O2	1024	18.012693	0.103988	0.11920
C -O3	256	0.099284	0.002016	0.33820
C -O3	512	2.026356	0.012204	0.13240
C -O3	1024	20.266728	0.166787	0.10580
NumPy	256	0.005137	0.003021	8.29360
NumPy	512	0.016657	0.009406	23.34500
NumPy	1024	0.030502	0.003577	71.24880
NumPy	2048	0.273489	0.008894	62.87160
NumPy, un hilo	256	0.002120	0.000419	16.35940
NumPy, un hilo	512	0.016846	0.001813	16.07000
NumPy, un hilo	1024	0.112902	0.003809	19.03840
NumPy, un hilo	2048	0.915865	0.032713	18.77660
Python puro	64	0.029421	0.001517	0.01786
Python puro	128	0.233313	0.011414	0.01802
Python puro	256	1.711266	0.081298	0.01964

Tabla 4. Resumen de los tiempos de ejecución, desviación estándar y rendimiento (GFLOPS) de las implementaciones en C, Python puro y NumPy para los tamaños de matriz evaluados.

Variante	n	Promedio (s)	Desv. estándar (s)	GFLOPS
C -O0	256	0.146838	0.012236	0.22980
C -O0	512	1.315971	0.016808	0.20420
C -O0	1024	26.504649	0.736820	0.08120
C -O2	256	0.078541	0.002225	0.42740
C -O2	512	1.843508	0.010402	0.14560
C -O2	1024	18.012693	0.103988	0.11920
C -O3	256	0.099284	0.002016	0.33820
C -O3	512	2.026356	0.012204	0.13240
C -O3	1024	20.266728	0.166787	0.10580
NumPy	256	0.005137	0.003021	8.29360
NumPy	512	0.016657	0.009406	23.34500
NumPy	1024	0.030502	0.003577	71.24880
NumPy	2048	0.273489	0.008894	62.87160
NumPy, un hilo	256	0.002120	0.000419	16.35940
NumPy, un hilo	512	0.016846	0.001813	16.07000
NumPy, un hilo	1024	0.112902	0.003809	19.03840
NumPy, un hilo	2048	0.915865	0.032713	18.77660
Python puro	64	0.029421	0.001517	0.01786
Python puro	128	0.233313	0.011414	0.01802
Python puro	256	1.711266	0.081298	0.01964

Tabla 5. Comparación del tiempo de ejecución y rendimiento de NumPy con todos los hilos disponibles y con un solo hilo para los tamaños de matriz evaluados.

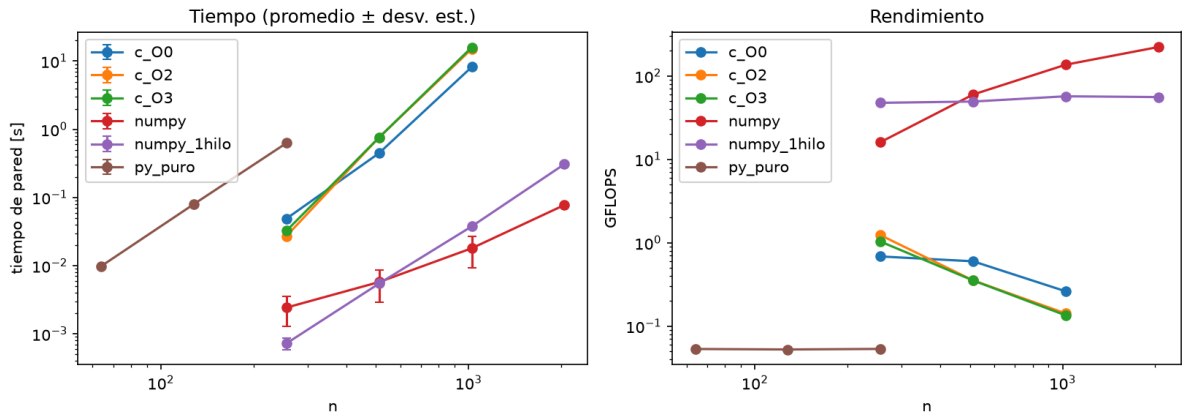


Figura 1. La gráfica izquierda presenta el tiempo promedio con desviación estándar y la derecha compara el rendimiento en GFLOPS.

En general, NumPy obtuvo un rendimiento superior al de las implementaciones en C y Python puro. La diferencia se relaciona con las rutinas numéricas optimizadas empleadas por NumPy. En C, el nivel de optimización no produjo una mejora uniforme para todos los tamaños, lo que hace necesario considerar las condiciones de ejecución y la variabilidad de las mediciones.

4. Análisis de las preguntas guía

4.1 Nivel de Optimización del compilador

Para (n=1024), la versión -O0 tardó 26.504649s y la versión -O3 -march=native tardó 20.266728s. Por tanto, la versión -O3 fue aproximadamente 1.31 veces más rápida.

La optimización del compilador puede transformar operaciones, reducir instrucciones innecesarias y utilizar instrucciones vectoriales. En el ensamblador generado se encontraron instrucciones vmulpd, vfmadd132sd y vfmdd231sd. Esto evidencia el uso de operaciones vectoriales y de multiplicación-suma, aunque no significa que todos los ciclos del algoritmo se ejecuten vectorizados.

La versión -O3 no fue la más rápida para todos los tamaños. Esto demuestra que un nivel de optimización más alto no garantiza siempre el menor tiempo en una medición particular.

4.2 Python puro contra NumPy

Para (n=256), Python puro tardó 1.711266 s, mientras que NumPy tardó 0.005137 s. En esta medición, NumPy fue aproximadamente 333 veces más rápido. La implementación de Python puro ejecuta numerosos ciclos mediante el intérprete. NumPy delega el cálculo en rutinas numéricas compiladas y optimizadas, que pueden aprovechar la vectorización, una organización de datos favorable a la caché y varios hilos de ejecución. Estas diferencias explican gran parte de la brecha observada.

4.3 Paralelismo con NumPy

Para (n=2048), NumPy en su configuración habitual tardó 0.273489 s, mientras que NumPy restringido a un hilo tardó 0.915865 s. La configuración habitual fue aproximadamente 3.35

veces más rápida.
La columna cpu_over_wall del archivo numpy.csv expresa la razón entre el tiempo de CPU y el tiempo de pared. Un valor superior a uno puede indicar que varios hilos acumularon tiempo de CPU simultáneamente. No debe interpretarse como un conteo exacto de hilos en todas las circunstancias.
Los resultados son consistentes con el aprovechamiento de paralelismo interno de la biblioteca numérica. Para obtener una línea base secuencial reproducible, es necesario limitar explícitamente los hilos, ya que la configuración habitual de NumPy puede ejecutar trabajo paralelo sin que el programador lo indique directamente.

4.4 Estimación del pico teórico

Se estimó el pico teórico de doble precisión mediante seis núcleos físicos, una frecuencia base de 2.1 GHz y 16 operaciones de punto flotante por ciclo y núcleo.
La mejor medición de C en la tabla fue 0.4274 GFLOPS, correspondiente a −02 con (n=256). Esto representa aproximadamente el 0.21 % del pico estimado. NumPy obtuvo 71.2488 GFLOPS con (n=1024), equivalente a aproximadamente el 35.3 % de esa estimación.

La implementación sencilla de multiplicación de matrices no necesariamente alcanza el pico teórico porque el rendimiento también depende del acceso a memoria, el orden de los ciclos, el aprovechamiento de las unidades vectoriales, las cachés, la frecuencia efectiva y los límites del entorno virtualizado. Además, la estimación emplea la frecuencia base y una tasa teórica aproximada; no representa el pico sostenido medido del procesador.

4.5 Variabilidad de las mediciones

La mayor desviación estándar relativa del resumen ocurrió en NumPy con (n=256):

(0.003021)/(0.005137)*100 = 58.8%

Esta variabilidad puede estar relacionada con el pequeño tiempo de ejecución, el costo de las llamadas, la actividad del sistema operativo y la variación de frecuencia del procesador. La medición se realizó dentro de WSL2, por lo que también deben considerarse las características del entorno virtualizado.

El resultado supera el umbral del 10 % recomendado en la práctica. Por ello, debe interpretarse con cautela y sería conveniente repetir esa configuración en condiciones controladas antes de utilizarla como una referencia precisa.

4.6 Comparación entre CPU y GPU

Se utilizó una NVIDIA Tesla T4 mediante CuPy en Google Colab.

Tamaño	CPU promedio (s)	GPU sin sincronizar (s)	GPU sincronizada (s)	Aceleración aprox.
256 × 256	0.000402	0.000097	0.000083	4.83×
4096 × 4096	1.540222	0.000156	0.038261	40.26×

Tabla 6. Mediciones de GPU.

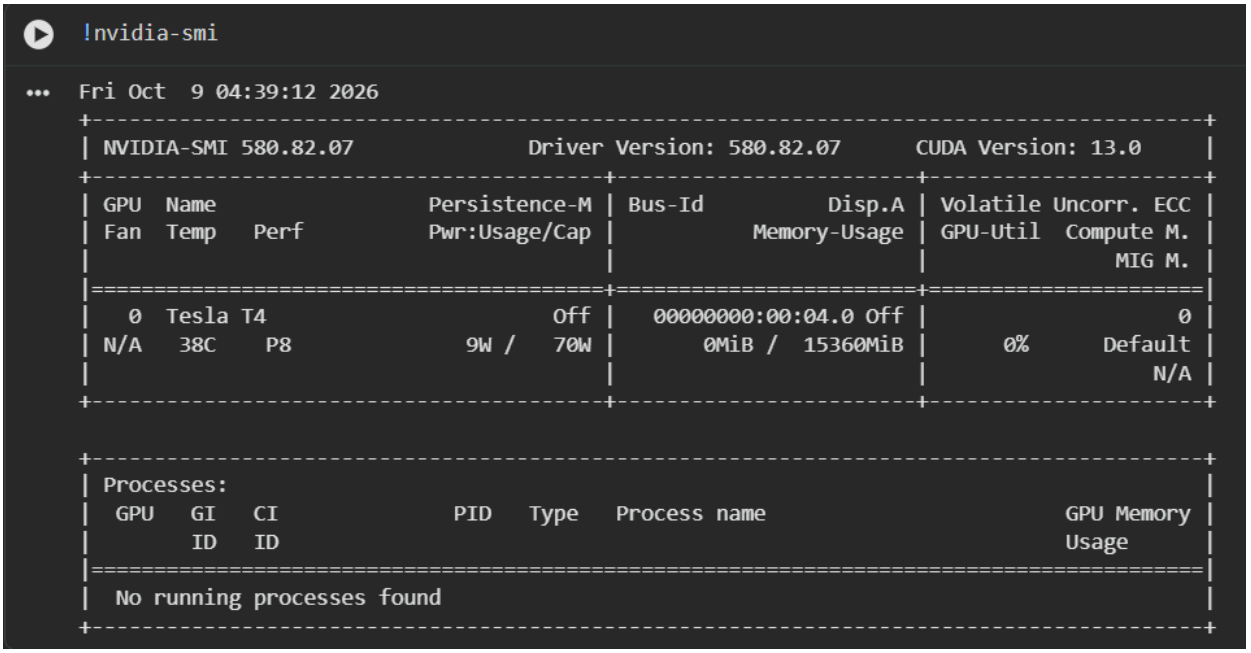


Imagen 1. Representación de NVIDIA Tesla T4 mediante CuPy en Google Colab.

En las mediciones obtenidas, la GPU fue más rápida para ambos tamaños. La aceleración aumentó de aproximadamente 4.83 veces para $n = 256$ a 40.26 veces para $n = 4096$. En matrices pequeñas, el envío de operaciones y la administración de datos pueden representar una parte importante del tiempo total, por lo que no se debe suponer que la GPU siempre ofrecerá una ventaja elevada.

La medición sin sincronización produjo tiempos artificialmente pequeños porque CuPy ejecuta operaciones de forma asíncrona. El programa puede continuar antes de que la GPU haya terminado la multiplicación. La sincronización obliga a esperar la finalización del trabajo, por lo que el tiempo sincronizado es el apropiado para estimar la duración real de la operación.

Los resultados de CPU y GPU deben compararse con cautela, porque se obtuvieron en entornos diferentes y las matrices de la parte de GPU utilizaron float32, mientras que la estimación de pico de la CPU se formuló en doble precisión.

5. GPU: evidencia y sincronización

La GPU asignada por Google Colab fue una NVIDIA Tesla T4. La herramienta nvidia-smi reportó el controlador 580.82.07, compatibilidad con CUDA 13.0 y aproximadamente 15 GB de memoria de GPU disponible.

Figura 2. Insertar la captura de la salida de nvidia-smi obtenida en Google Colab.

La comparación muestra que la GPU puede ofrecer ventajas importantes en operaciones matriciales grandes. Sin embargo, el tiempo medido depende del

tamaño de la matriz, la precisión numérica, las operaciones de transferencia y la sincronización. Para que el cronómetro refleje la finalización del cálculo, se utilizó `cp.cuda.Stream.null.synchronize()` antes de detener la medición.

6. Problemas encontrados y soluciones

Durante la preparación se presentaron los siguientes problemas:

- **Disponibilidad de perf en WSL2.** La herramienta no estaba disponible para el kernel utilizado. Se continuó con la medición mediante relojes de alta resolución y herramientas de tiempo alternativas.
- **Instalación de CuPy.** Inicialmente aparecieron advertencias por instalaciones de CuPy incompatibles y posteriormente un error por la ausencia de libcublas.so.12. Se reinstaló CuPy con las dependencias correspondientes y se verificó la multiplicación en la Tesla T4.
- **Variabilidad de las mediciones.** Algunas configuraciones de NumPy, especialmente con matrices pequeñas, presentaron desviaciones relativas elevadas. Se registraron los promedios y desviaciones para no ocultar esta limitación.
- **Control del paralelismo.** Se ejecutó NumPy tanto en su configuración habitual como restringiendo las variables de entorno a un hilo, lo que permitió observar el efecto del paralelismo interno.

5. Conclusiones

Mediante el desarrollo del trabajo se logró establecer un entorno de experimentación equipado con GCC, Python, NumPy y OpenMP sobre WSL2, evaluando adicionalmente el comportamiento de una GPU NVIDIA Tesla T4 a través de la plataforma Google Colab. La evidencia experimental constató que NumPy brinda un rendimiento significativamente superior frente a Python puro, impulsado por sus bibliotecas numéricas altamente optimizadas. En cuanto a las banderas de compilación en C, las fluctuaciones obtenidas demuestran que elevar la optimización al máximo nivel no asegura de forma invariable el tiempo mínimo. De igual manera, el contraste entre CPU y GPU evidenció incrementos de velocidad más notorios al procesar dimensiones matriciales de gran volumen. Por último, se confirmó que la sincronización resulta esencial para capturar la duración real en cómputo asíncrono, mientras que la dispersión estadística de los datos recalca la relevancia de efectuar corridas reiteradas en condiciones estrictamente controladas.

6. Uso de inteligencia artificial

Se empleó ChatGPT como asistente conceptual para la interpretación de los requerimientos, la estructuración general del documento, el procesamiento de las métricas numéricas y la depuración de fallos surgidos durante la fase de instalación. Todo el conjunto de pruebas, registros temporales, desviaciones y capturas del estado de la GPU mediante `nvidia-smi` provienen directamente de ejecuciones reales en las plataformas utilizadas. Dichos insumos deben preservarse junto con los archivos numéricos generados a fin de respaldar la validez de los experimentos.

7. Contribución individual

La siguiente tabla desglosa la aportación y tareas desarrolladas por cada uno de los miembros del equipo. El porcentaje global debe totalizar exactamente el 100 %.

Integrante	Actividades realizadas	Contribución
Gónzalez Gónzalez Alan	Configuración del entorno de trabajo y ejecuciones locales iniciales	[33.3%]
Flores Lopez Mariana	Toma de tiempos, procesamiento de métricas estadísticas y graficación	[33.3 %]
Pérez Guzmán Shaiel Aylin	Pruebas de aceleración en GPU, redacción final y revisión del reporte	[33.3%]
Total		**100 %**